

Algoritmos y Estructuras de Datos I

Arreglos

Damián Barsotti

Fa.M.A.F., Universidad Nacional de Córdoba, [Argentina](#)

Arreglos

Características

Arreglos

Características

- Almacena elementos de un mismo tipo.

Arreglos

Características

- Almacena elementos de un mismo tipo.
- Análogo a las listas en programación funcional.

Arreglos

Características

- Almacena elementos de un mismo tipo.
- Análogo a las listas en programación funcional.
- Se representa como función definida sobre un segmento de los naturales.

Arreglos

Características

- Almacena elementos de un mismo tipo.
- Análogo a las listas en programación funcional.
- Se representa como función definida sobre un segmento de los naturales.
- Es el tipo de datos **contenedor** más utilizado en programación imperativa.

Arreglos

Características

- Almacena elementos de un mismo tipo.
- Análogo a las listas en programación funcional.
- Se representa como función definida sobre un segmento de los naturales.
- Es el tipo de datos **contenedor** más utilizado en programación imperativa.

Declaración

$a : \text{array } [p, q) \text{ of } A$

donde $p \leq q$ y A indica el tipo de los elementos.

Arreglos

Características

- Almacena elementos de un mismo tipo.
- Análogo a las listas en programación funcional.
- Se representa como función definida sobre un segmento de los naturales.
- Es el tipo de datos **contenedor** más utilizado en programación imperativa.

Declaración

$a : \text{array } [p, q) \text{ of } A$

donde $p \leq q$ y A indica el tipo de los elementos.

Ejemplos

$a : \text{array } [0, 4) \text{ of } \text{Int} \quad a = [3, -1, 7, 0]$

$a : \text{array } [5, 7) \text{ of } \text{Bool} \quad a = [\text{True}, \text{False}]$

Acceso a elementos

a : array [0, 4) of Int, *a* = [3, -1, 7, 0]

a.0 = 3 *a*.1 = -1 *a*.2 = 7 *a*.3 = 0

Acceso a elementos

a : array [0, 4) of Int, a = [3, -1, 7, 0]

a.0 = 3 a.1 = -1 a.2 = 7 a.3 = 0

a : array [5, 7) of Bool, a = [True, False]

a.5 = True a.6 = False

Propiedades

- La **cantidad de elementos** es $q - p$. Si $q = p$ el arreglo es **vacío**.

Propiedades

- La **cantidad de elementos** es $q - p$. Si $q = p$ el arreglo es **vacío**.
- No es necesario que el primer índice sea 0.

Propiedades

- La **cantidad de elementos** es $q - p$. Si $q = p$ el arreglo es **vacío**.
- No es necesario que el primer índice sea 0.
- Un **elemento** del arreglo a se referencia por $a.n$, donde n solo tiene sentido en el rango $p \leq n < q$.

Propiedades

- La **cantidad de elementos** es $q - p$. Si $q = p$ el arreglo es **vacío**.
- No es necesario que el primer índice sea 0.
- Un **elemento** del arreglo a se referencia por $a.n$, donde n solo tiene sentido en el rango $p \leq n < q$.
- Decimos que un valor v **está en** el arreglo si algún elemento es igual a v .

Propiedades

- La **cantidad de elementos** es $q - p$. Si $q = p$ el arreglo es **vacío**.
- No es necesario que el primer índice sea 0.
- Un **elemento** del arreglo a se referencia por $a.n$, donde n solo tiene sentido en el rango $p \leq n < q$.
- Decimos que un valor v **está en** el arreglo si algún elemento es igual a v .
- En el teorico van a ser constantes:

Propiedades

- La **cantidad de elementos** es $q - p$. Si $q = p$ el arreglo es **vacío**.
- No es necesario que el primer índice sea 0.
- Un **elemento** del arreglo a se referencia por $a.n$, donde n solo tiene sentido en el rango $p \leq n < q$.
- Decimos que un valor v **está en** el arreglo si algún elemento es igual a v .
- En el teorico van a ser constantes:
 - `Var  $a$  : array [0, 4) of Int;`
donde a no cambia, o

Propiedades

- La **cantidad de elementos** es $q - p$. Si $q = p$ el arreglo es **vacío**.
- No es necesario que el primer índice sea 0.
- Un **elemento** del arreglo a se referencia por $a.n$, donde n solo tiene sentido en el rango $p \leq n < q$.
- Decimos que un valor v **está en** el arreglo si algún elemento es igual a v .
- En el teorico van a ser constantes:
 - $\text{Var } a : \text{array } [0, 4) \text{ of } \text{Int};$
donde a no cambia, o
 - $\text{Const } A : \text{array } [0, 4) \text{ of } \text{Int};$

Propiedades

- La **cantidad de elementos** es $q - p$. Si $q = p$ el arreglo es **vacío**.
- No es necesario que el primer índice sea 0.
- Un **elemento** del arreglo a se referencia por $a.n$, donde n solo tiene sentido en el rango $p \leq n < q$.
- Decimos que un valor v **está en** el arreglo si algún elemento es igual a v .
- En el teorico van a ser constantes:
 - `Var  $a$  : array [0, 4) of Int;`
donde a no cambia, o
 - `Const  $A$  : array [0, 4) of Int;`
- Es posible usar **variables cuantificadas** sobre arreglos.

Cuantificadores sobre arreglos

Ejemplo

Si $a : \text{array } [p, q) \text{ of } \text{Int}$, entonces:

$$\langle \text{Max } i : p \leq i < q : a.i \rangle$$

indica el **máximo elemento** del arreglo.

Cuantificadores sobre arreglos

Ejemplo

Si $a : \text{array } [p, q) \text{ of } \text{Int}$, entonces:

$$\langle \text{Max } i : p \leq i < q : a.i \rangle$$

indica el **máximo elemento** del arreglo.

Otro ejemplo

$$\langle \sum i : 0 \leq i < N : a.i \rangle$$

indica la **suma de todos los elementos** de un arreglo

$a : \text{array } [0, N) \text{ of } \text{Int}$.

Cuantificadores sobre arreglos

Ejemplo

Si $a : \text{array } [p, q) \text{ of } \text{Int}$, entonces:

$$\langle \text{Max } i : p \leq i < q : a.i \rangle$$

indica el **máximo elemento** del arreglo.

Otro ejemplo

$$\langle \sum i : 0 \leq i < N : a.i \rangle$$

indica la **suma de todos los elementos** de un arreglo

$a : \text{array } [0, N) \text{ of } \text{Int}$.

Segmento

$$\langle \sum i : 0 \leq i < n : a.i \rangle \quad \text{con } 0 \leq n \leq N$$

indica la suma de los primeros n elementos (un **segmento** del arreglo).

Implementación en memoria

Abstracción

Implementación en memoria

Abstracción

- Lo visto es una **representación matemática**.

Implementación en memoria

Abstracción

- Lo visto es una **representación matemática**.
- Hardware: **casillas contiguas** de memoria RAM.

Var a : array $[0, 4)$ of Int ;
 n : Int ;

<i>Memoria</i>	
$a.0$:	3
$a.1$:	-1
$a.2$:	7
$a.3$:	0
n :	4
	$\vdots$

Implementación en memoria

Abstracción

- Lo visto es una **representación matemática**.
- Hardware: **casillas contiguas** de memoria RAM.

Consecuencias

Var a : array $[0, 4)$ of Int ;
 n : Int ;

<i>Memoria</i>	
$a.0$:	3
$a.1$:	-1
$a.2$:	7
$a.3$:	0
n :	4
	$\vdots$

Implementación en memoria

Abstracción

- Lo visto es una **representación matemática**.
- Hardware: **casillas contiguas** de memoria RAM.

Consecuencias

- **Eficiente:** acceso y modificación de elementos en tiempo constante.

Var a : array $[0, 4)$ of Int ;
 n : Int ;

<i>Memoria</i>	
$a.0$:	3
$a.1$:	-1
$a.2$:	7
$a.3$:	0
n :	4
	$\vdots$

Implementación en memoria

Abstracción

- Lo visto es una **representación matemática**.
- Hardware: **casillas contiguas** de memoria RAM.

Consecuencias

- **Eficiente:** acceso y modificación de elementos en tiempo constante.
- **Poco eficiente:** agregar elementos si no hay espacio contiguo suficiente — hay que **copiar todo el arreglo** a otro bloque de memoria donde quepa.

Var a : array $[0, 4)$ of Int ;
 n : Int ;

Memoria	
$a.0:$	3
$a.1:$	-1
$a.2:$	7
$a.3:$	0
$n:$	4
	$\vdots$